

第9章 并发与异常

建议学时:6-10 小时 | 难度:★★★★★(全书最高) | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 理解 goroutine 与 C 线程的本质差异:pthread_create 到 go 关键字
- 掌握 CSP 模型:channel 的发送/接收语义,有缓冲与无缓冲的区别
- 掌握通道关闭的规范:close、ok 断言、for range 消费
- 掌握 select:多通道多路复用、超时与 default、随机公平性
- 掌握 sync 全家桶:Mutex/RWMutex/WaitGroup/Once/atomic 与适用场景
- 掌握数据竞争与 race detector:为什么 -race 是标配
- 掌握 Go 的"异常"哲学:error 接口、errors.Is/As/Join、panic/recover
- 掌握 context:取消信号与超时在 goroutine 树中传递

💡 从 C 到 Go:本章主题如何迁移

C 的并发是"操作系统线程 + 手工互斥": pthread_create 建线程(默认 8MB 栈,创建慢), pthread_mutex_lock 手工保护临界区,线程间的通信要么共享内存 + 加锁,要么手工实现队列;一个失误就是死锁、数据竞争、段错误。教科书还会提醒你"线程太多切换开销大",于是 Linux 上还有 epoll 事件循环这种反直觉的写法。

Go 的答案完全不同:goroutine 是**语言级轻量线程**(初始几 KB 栈,自动增长),创建十万个也毫无压力;线程间通信的首选是 **channel**——"不要通过共享内存来通信,而要通过通信来共享内存",CSP 模型让数据流动的方向一目了然;go vet 之外的 go run -race 会把数据竞争当场抓住;错误则彻底告别异常:函数显式返回 error, errors.Is/As/Join 提供比 C errno 精细得多的错误链。

一句话主线: C 的并发是"操作系统 API + 心智负担",Go 的并发是"语言内置 + 编译器保镖"——本章是全书最重要的一章,值得反复读三遍。

📦 前置知识自查

- C 的 pthread_create/join/mutex/semaphore 基本用法(会对照即可)
- 第 4 章:函数值、闭包(goroutine 的本质是闭包)
- 第 5 章:goroutine 泄漏(\$5.5,本章的 select/default 是其解药)
- 第 8 章:goroutine per connection 已经隐式使用过并发
- 进程与线程、临界区、死锁的基本概念

🚀 本章学习路线

1. **第一阶段**: \$9.1-\$9.3, goroutine + channel 亲手跑通(先忘掉锁)
2. **第二阶段**: \$9.4-\$9.6, select、sync、race detector, 把"并发三件套"凑齐
3. **第三阶段**: \$9.7-\$9.9, error 链、panic/recover、context, 补齐"异常"部分
4. **验收标准**: 能默写"worker pool + select 超时 + context 取消 + WaitGroup"的完整骨架

本章知识导图

- §9.1 goroutine:pthread_create → go 关键字,轻量线程的真相
- §9.2 channel:CSP 通信,有缓冲与无缓冲的阻塞语义
- §9.3 通道关闭:close、ok 断言、for range 消费
- §9.4 select:多路复用、超时与 default、随机公平
- §9.5 sync:Mutex/RWMutex/WaitGroup/Once/atomic
- §9.6 数据竞争:go run -race 与竞争修复
- §9.7 error 哲学:errors.Is/As/Join 错误链
- §9.8 panic/recover:错误处理最后手段
- §9.9 context:取消与超时沿 goroutine 树传递

§9.1 goroutine:pthread_create 的终结者

C 的写法	Go 的写法
<code>pthread_create(&tid, NULL, worker, arg)</code>	<code>go worker(arg)</code> — 关键字,不是库函数
线程默认栈 8MB,创建与切换昂贵	goroutine 初始栈约 2-8KB,按需自动增长,十万并发是日常
<code>pthread_join(tid, &ret)</code>	不需要;goroutine 执行完即消失
线程由操作系统调度(内核线程)	goroutine 由 Go 运行时 M:N 调度到少量 OS 线程上
数据竞争 = 未定义行为,只能靠锁纪律	数据竞争被 race detector 检测,CI 标配 -race
线程间通信:共享内存 + 锁(教科书警告不要裸奔)	首选 channel 通信(CSP),锁降级为必要时的补充

示例 9-1 一百万个 goroutine

```

package main

import (
    "fmt"
    "sync"
)

func main() {
    var wg sync.WaitGroup

    // 一百万并发:每个 goroutine 只占几 KB 栈
    // 同样的数量用 pthread_create 会直接 OOM
    for i := 0; i < 1_000_000; i++ {
        wg.Add(1)
        go func(n int) {
            defer wg.Done()
            _ = n * n
        }(i)
    }

    wg.Wait()
    fmt.Println("一百万个 goroutine 全部完成")
}

```

⚠ 易错点 9-1:闭包捕获循环变量

`go func() { fmt.Println(i) }()` 直接捕获循环变量 `i`:Go 1.22 之前所有 goroutine 看到的都是同一个 `i`(打印全是一样的值);Go 1.21 及更早的修复方式是传参 `go func(n int){...}(i)`。Go 1.22+ 循环变量每轮独立,但**传参**依然是更清晰的习惯。

§9.2 channel:CSP 通信

channel 是有类型的通信管道: `ch <- v` 发送, `v := <-ch` 接收。**无缓冲通道**的发送与接收必须同时就绪,否则一方阻塞等待另一方——这既是同步点又是数据传递;**有缓冲通道**在缓冲区满之前不阻塞发送,在缓冲为空时不阻塞接收。

■ 示例 9-2 无缓冲 channel:同步 + 传值

```

package main

import (
    "fmt"
    "time"
)

func main() {
    ch := make(chan string) // 无缓冲

    go func() {
        time.Sleep(100 * time.Millisecond) // 模拟准备工作
        ch <- "计算结果" // 发送:阻塞直到主 goroutine 来接收
        fmt.Println("发送完成")
    }()

    // 主 goroutine 阻塞等待,收到数据才继续
    v := <-ch
    fmt.Println("收到:", v)
}

```

示例 9-3 有缓冲 channel:生产者消费者

```

package main

import (
    "fmt"
)

func main() {
    jobs := make(chan int, 3) // 缓冲 3 个,先攒后取

    // 生产者:一次放 5 个任务
    go func() {
        for i := 1; i <= 5; i++ {
            jobs <- i
            fmt.Printf("放入任务 %d\n", i)
        }
        close(jobs) // 放完必须关闭,消费者才能结束
    }()

    // 消费者:for range 直到通道关闭
    for j := range jobs {
        fmt.Printf("处理任务 %d\n", j)
    }
    fmt.Println("所有任务处理完毕")
}

```

★ 重点:channel 语义速记

- 发送到已关闭的通道:panic(生产者唯一的关闭者,绝不双关)
- 从已关闭的空通道接收:立即返回零值,不阻塞(需要 ok 断言区分)
- `v, ok := <-ch`:ok 为 false 表示通道已关闭且已取空
- 对 nil 通道收发:永久阻塞(常被用于 select 中"禁用某分支")
- 通道所有权:生产者负责关闭,消费者只读不关

§9.3 通道关闭与 range 消费

示例 9-4 关闭的完整规范

```
package main

import (
    "fmt"
)

func main() {
    ch := make(chan int, 5)

    go func() {
        defer close(ch) // 生产者负责关闭
        for i := 0; i < 5; i++ {
            ch <- i * i
        }
    }()

    for {
        v, ok := <-ch
        if !ok {
            fmt.Println("通道已关闭,结束")
            break
        }
        fmt.Println("取到:", v)
    }
    // 等价写法:for v := range ch { ... }
}
```

⚠ 易错点 9-2:向已关闭通道发送会 panic

生产中 90% 的 channel panic 来自"双关闭"或"关闭后继续发送"。纪律:①关闭者是的发送方(通常用 defer 放在生产者函数开头);②接收方永远不关闭;③不确定是否关闭时用"关闭信号 channel"或 sync.Once 包一层。

§9.4 select:多路复用与超时

select 同时监听多个通道,任一就绪即执行对应分支;多个同时就绪时**随机**选择一个(公平);没有就绪且没有 default 时阻塞;default 使 select 变为"非阻塞轮询"。它是 C 里 select/epoll 的语法级替代,也是"超时"的天然实现。

示例 9-5 select:超时、优先与关闭信号

```

package main

import (
    "fmt"
    "time"
)

func main() {
    data := make(chan int)
    quit := make(chan struct{}) // 空结构体通道:纯信号

    // 工作 goroutine:收到 quit 就退出
    go func() {
        for {
            select {
            case v := <-data:
                fmt.Println("处理数据:", v)
            case <-quit:
                fmt.Println("收到退出信号,结束")
                return
            case <-time.After(2 * time.Second):
                fmt.Println("两秒无数据,做点别的")
            }
        }
    }()

    // 主 goroutine 喂数据后发退出信号
    for i := 0; i < 3; i++ {
        data <- i
        time.Sleep(100 * time.Millisecond)
    }
    close(quit)
    time.Sleep(300 * time.Millisecond)
}

```

| 示例 9-6 非阻塞发送:default 分支

```

package main

import (
    "fmt"
)

func main() {
    ch := make(chan int, 1)

    // 满则跳过:绝不阻塞
    select {
    case ch <- 1:
        fmt.Println("已放入")
    default:
        fmt.Println("缓冲区满,丢弃本次")
    }

    select {
    case ch <- 2:
        fmt.Println("已放入")
    default:
        fmt.Println("缓冲区满,丢弃本次")
    }
}

```

工程建议:用 select 做"可取消的发送"

向一个可能无人接收的通道发送时,配一个 select 防死锁:退出信号通道一旦关闭,发送分支不再阻塞,goroutine 可以优雅退场。这也是第 5 章 goroutine 泄漏的标准解药之一:发送不出去的 goroutine + select quit = 可以安全取消。

§9.5 sync 全家桶:Mutex、WaitGroup、Once、atomic

channel 适合"数据流动"的场景;而"共享计数器""初始化一次""等待一组任务"这类场景,sync 包是更直接的答案——C 的 pthread_mutex/pthread_once 都有对应物,且语法更干净。

示例 9-7 互斥锁与原子操作

```

package main

import (
    "fmt"
    "sync"
    "sync/atomic"
)

func main() {
    var mu sync.Mutex
    var counter int // 普通变量:需要锁保护

    var atom atomic.Int64 // 原子变量:免锁

    var wg sync.WaitGroup
    for i := 0; i < 1000; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            mu.Lock() // 临界区入口
            counter++
            mu.Unlock() // 临界区出口(生产用 defer)
            atom.Add(1)
        }()
    }
    wg.Wait()

    fmt.Println("锁保护计数:", counter) // 必然是 1000
    fmt.Println("原子计数:", atom.Load()) // 必然是 1000
}

```

示例 9-8 sync.Once:只执行一次


```

package main

import (
    "fmt"
    "sync"
)

var (
    config    string
    configMu sync.Once // 只会执行一次,并发安全
)

// getConfig 任意 goroutine 并发调用,初始化只发生一次
func getConfig() string {
    configMu.Do(func() {
        fmt.Println("加载配置...")
        config = "prod-2026"
    })
    return config
}

func main() {
    var wg sync.WaitGroup
    for i := 0; i < 10; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            fmt.Println(getConfig())
        }()
    }
    wg.Wait() // "加载配置..." 只打印一次
}

```

★ 重点:同步原语选择表

- **数据流动** → channel(生产/消费、流水线)
- **临界区读写共享变量** → Mutex;读多写少 → RWMutex
- **等待一组 goroutine 完成** → WaitGroup(注意:Add 必须在启动前,不能并发 Add 后 Wait 误配)
- **初始化只做一次** → Once(替代 C 的 pthread_once)
- **单个 int64/指针的增减读取** → atomic(免锁,无竞争)
- **禁止拷贝**:sync 类型(含 channel 内含)不可复制,传递一律用指针或引用

§9.6 数据竞争与 race detector

两个 goroutine 同时读写同一个变量,且至少一个是写,称为数据竞争(data race)。C 中它是"未定义行为",可能产生任何结果;Go 在运行时用 **race detector** 标记它: `go run -race main.go` 或 `go test -race ./...`。它基于 C/C++ 的 ThreadSanitizer 技术,只增加约 2-10 倍运行时开销,是 CI 的标配。

示例 9-9 竞争复现与修复

```

package main

import (
    "fmt"
    "sync"
)

func main() {
    // 版本 A:无保护的共享计数器 → go run -race 会报警告
    var total int
    var wg sync.WaitGroup
    for i := 0; i < 1000; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            total++ // 数据竞争:读改写不是原子的
        }()
    }
    wg.Wait()
    fmt.Println("竞争版本结果(可能是 1000,也可能不是):", total)

    // 版本 B:加锁修复 → -race 无警告,结果必为 1000
    var safe int
    var mu sync.Mutex
    for i := 0; i < 1000; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            mu.Lock()
            safe++
            mu.Unlock()
        }()
    }
    wg.Wait()
    fmt.Println("加锁版本结果:", safe)
}

```

⚠ 易错点 9-3:race 检测不到一切,但它说有的必有

race detector 只对**确实发生过的**交错采样报告:没被触发就检测不到(所以要在压测下跑)。但反过来——它报告了,就一定是真的竞争。它也不检测 channel 使用错误(如向已关闭通道发送),那会直接 panic。纪律:所有测试用 `go test -race ./...`,发布前跑一遍。

§9.7 error 哲学:显式错误链

Go 没有异常。函数把错误作为返回值显式传递: `f, err := os.Open(...)` ——错误是"值",可以比较、包装、传递、记录。三个 API 组成错误工具箱:

- `fmt.Errorf("...: %w", err)`:包装错误,保留原错误链
- `errors.Is(err, target)`:判断链中是否存在某个哨兵错误(替代 C 的 `errno` 比较)
- `errors.As(err, &target)`:把链中某个类型化错误取出(替代 C 的 `errno` 到结构体的手写转换)

- `errors.Join(errs...)` (1.20+):聚合多个错误为一个(如并行任务的部分失败)

示例 9-10 哨兵错误与错误链

```
package main

import (
    "errors"
    "fmt"
    "os"
)

// 哨兵错误:相当于 C 的 errno 常量
var ErrNotFound = errors.New("记录不存在")

// FindUser 模拟数据库查询:找不到返回哨兵错误
func FindUser(id int) error {
    if id != 42 {
        return fmt.Errorf("查询用户 %d: %w", id, ErrNotFound)
    }
    return nil
}

func main() {
    err := FindUser(7)
    if err == nil {
        fmt.Println("找到了")
        return
    }
    // errors.Is 沿 %w 链查找:即使被多层包装也能判断
    if errors.Is(err, ErrNotFound) {
        fmt.Println("是\"记录不存在\"类错误:", err)
    }
    fmt.Println("err != nil,必须处理(不处理编译不通过的事实强迫你思考)")

    // 顺便演示:经典判错模式
    f, err := os.Open("不存在的文件.txt")
    if err != nil {
        if errors.Is(err, os.ErrNotExist) {
            fmt.Println("文件不存在:", err)
            return
        }
        fmt.Println("其他错误:", err)
        return
    }
    defer f.Close()
}
```

示例 9-11 errors.Join 与 errors.As

```

package main

import (
    "errors"
    "fmt"
)

type TimeoutErr struct {
    Op string
}

func (e *TimeoutErr) Error() string {
    return "操作超时: " + e.Op
}

func main() {
    // errors.Join:聚合多个错误
    var errs []error
    for _, op := range []string{"连接", "读取", "写入"} {
        errs = append(errs, &TimeoutErr{Op: op})
    }
    joined := errors.Join(errs...)
    fmt.Println(joined)

    // errors.As:取出链中特定类型的错误
    var te *TimeoutErr
    if errors.As(joined, &te) {
        fmt.Printf("链中第一个超时错误:%s\n", te.Op)
    }
}

```

★ 重点:错误处理铁律

- 错误只向上传递一层:能处理就地处理(如重试、降级),不能处理就 %w 包装后上交,保留上下文
- 函数签名必须把错误放在返回值里,禁止静默吞掉;err == nil 必须意味着成功
- 哨兵错误用 errors.Is 判断,类型错误用 errors.As 取用,绝不直接 == 比较包装过的错误

§9.8 panic/recover:错误处理的最后手段

panic 是 Go 的"运行时致命错误"(越界、除零、断言失败)与"不可恢复错误"(如配置缺失)的出口:触发后函数立即停止,defer 逆序执行,沿调用栈向上传播,直到 main——若无人 recover,程序崩溃退出(栈打印)。recover 只能在 defer 函数中调用,把 panic 拦截为普通的返回值,是"让服务不因单请求而宕机"的机制(如 HTTP 服务器对每个请求做 panic 恢复)。

示例 9-12 panic/recover 拦截

```

package main

import (
    "fmt"
)

// safeDivide 用 recover 把 panic 转成返回值:生产网关式写法
func safeDivide(a, b int) (result int, err error) {
    defer func() {
        if r := recover(); r != nil {
            err = fmt.Errorf("内部错误: %v", r)
        }
    }()
    if b == 0 {
        panic("除数为零") // 业务上不该发生,视为编程错误
    }
    return a / b, nil
}

func main() {
    if r, err := safeDivide(10, 0); err != nil {
        fmt.Println("调用失败:", err)
    } else {
        fmt.Println("结果:", r)
    }
    fmt.Println("程序继续运行,没有被崩溃")
}

```

⚠ 易错点 9-4:recover 不是 try/catch

recover 只能捕获**本 goroutine**的 panic:别的 goroutine panic 会直接崩掉整个程序(包括 main)。所以"每个 goroutine 入口都应考虑 recover"(生产框架帮你做了,如 net/http 的服务器会恢复 handler 的 panic 并把连接断掉)。recover 也不接受异常类型过滤——捕获后要自己分类。绝不把 panic 当普通错误流使用,panic 只用于"不该发生的编程错误"。

§9.9 context:取消与超时沿树传递

context 解决"如何取消一串 goroutine":一个请求进来,开出 5 个 goroutine;客户端断开,这 5 个 goroutine 都要停。做法:context 作为函数第一个参数传递,子 goroutine 通过 `ctx.Done()` 通道感知取消;defer cancel() 保证资源释放。三个构造器:Background(根)、WithCancel、WithTimeout/WithDeadline。

示例 9-13 context 取消传播

```

package main

import (
    "context"
    "fmt"
    "time"
)

// worker 子任务:监听 ctx.Done() 实现可取消
func worker(ctx context.Context, id int, ch chan<- string) {
    select {
    case <-time.After(3 * time.Second): // 模拟耗时任务
        ch <- fmt.Sprintf("worker%d 完成", id)
    case <-ctx.Done(): // 父级取消:立即停手
        fmt.Printf("worker%d 被取消:%v\n", id, ctx.Err())
        return
    }
}

func main() {
    // 2 秒后自动取消:模拟"客户端等不及走了"
    ctx, cancel := context.WithTimeout(context.Background(), 2*time.Second)
    defer cancel()

    ch := make(chan string, 3)
    for i := 0; i < 3; i++ {
        go worker(ctx, i, ch)
    }

    // 等任一结果或超时
    select {
    case msg := <-ch:
        fmt.Println(msg)
    case <-ctx.Done():
        fmt.Println("主流程:全部取消,退出")
    }
}

```

工程建议:context 四原则

①函数第一个参数是 context(标准库 net/http 的 Request 自带 ctx);②永远不把 nil 传给需要 ctx 的地方,不确定就传 context.Background();③context 只携带取消/超时/元数据,不携带业务参数;④defer cancel() 紧跟 WithCancel,防止泄漏——漏掉 cancel 的 WithCancel 是 goroutine 泄漏的常见来源。

本章速查表

主题	要点
goroutine	<code>go f(args)</code> 启动;轻量线程,几 KB 栈,十万级无压力
channel	<code>make(chan T)</code> 无缓冲; <code>make(chan T, n)</code> 有缓冲;发送 <code><-</code> 、接收 <code><-ch</code>
关闭	生产者 <code>close</code> ;向已关闭通道发送 <code>panic</code> ; <code>v, ok := <-ch</code> 判断关闭
消费	<code>for v := range ch</code> 直到关闭自动退出
select	多通道多路复用;同时就绪随机选; <code>default</code> 非阻塞;可做超时
Mutex	临界区; <code>Lock/Unlock</code> (生产配 <code>defer</code>); <code>RWMutex</code> 读多写少
WaitGroup	<code>Add</code> (启动前)/ <code>Done</code> / <code>Wait</code> 等待一组 goroutine
Once	<code>Do</code> 回调只执行一次,并发安全(初始化单例)
atomic	<code>atomic.Int64</code> 等免锁读写单个变量; <code>Load/Add/Store/CompareAndSwap</code>
race	<code>go run -race</code> / <code>go test -race</code> 检测数据竞争
error 链	<code>fmt.Errorf(%w)</code> 包装; <code>errors.Is</code> 哨兵; <code>errors.As</code> 取类型; <code>errors.Join</code> 聚合
panic/recover	<code>panic</code> 编程错误; <code>recover</code> 只能在 <code>defer</code> 里;仅本 goroutine 有效
context	<code>Background/WithCancel/WithTimeout</code> ; <code>ctx.Done()</code> 取消; <code>defer cancel()</code>

最小必会清单

- 能默写 `go + WaitGroup` 模板与 `channel` 生产消费模板
- 能默写 `select` 超时与 `default` 非阻塞发送模板
- 能解释无缓冲通道与有缓冲通道的阻塞差异,并用代码验证
- 能默写"通道关闭三规则":生产者关、`defer` 关、接收方不关
- 能默写 `Mutex` 保护计数器的完整代码,并知道为什么 `-race` 无警告
- 能写出 `errors.Is` 判断哨兵错误的完整模式
- 能写出 `defer + recover` 把 `panic` 转为 `error` 的网关模式
- 能写出 `WithTimeout + ctx.Done() + defer cancel` 的可取消任务模板

自测题

Q 1. 无缓冲通道 `ch <- 1` 在没有人接收时会发生什么?如何让它"不阻塞地尝试发送"?

✅ 参考答案

发送 goroutine 永久阻塞,直到有接收方就绪(这是同步语义,也常是 goroutine 泄漏的根源,见第 5 章)。`"尝试发送"`用 `select + default` 或 `select + quit` 通道:`default` 分支立即执行而不阻塞;`quit` 分支让发送方在退出信号到来时解除阻塞优雅退出。

Q 2. 下面的代码输出什么?为什么?

✅ 参考答案

```
ch := make(chan int, 2)
ch <- 1
ch <- 2
close(ch)
v, ok := <-ch
fmt.Println(v, ok)
v, ok = <-ch
fmt.Println(v, ok)
v, ok = <-ch
fmt.Println(v, ok)
```

输出:1 true;2 true;0 false。前两次从缓冲取到值,ok 为 true;第三次缓冲已空且通道已关闭,返回零值 0 且 ok 为 false——这是"通道已取空"的可靠信号,for range 正是依赖它结束循环。

Q 3. C 的 pthread_create 与 Go 的 go 关键字在资源与调度上有何本质区别?

✅ 参考答案

pthread 创建内核线程:默认 8MB 栈、进入内核、随线程数线性增加调度负担,数千个就可能压垮系统;goroutine 由运行时管理:初始几 KB 栈按需增长,M:N 调度器把大量 goroutine 映射到少量 OS 线程,创建切换都在用户态,十万并发是常态。代价是 goroutine 内的阻塞式系统调用仍会占用 OS 线程(运行时做了 netpoll 优化,网络 I/O 不占线程)。

Q 4. 什么是数据竞争?为什么 C 的竞争是"未定义行为"而 Go 能抓住?如何让 Go 抓住?

✅ 参考答案

两个 goroutine 并发访问同一变量且至少一个是写。C 中竞争破坏内存模型,结果不可预测(可能崩溃、可能错值、可能看似正常);Go 的 race detector(基于 ThreadSanitizer)在运行时插桩所有内存访问,检测到实际发生的竞争交错就报告。用法:go run -race 或 go test -race ./...。注意它只报告"发生过"的竞争,所以要配合压测。

Q 5. 说出 errors.Is 与 errors.As 的适用场景差异,并各举一例。

✅ 参考答案

Is 判断"错误链中是否存在某个哨兵值":如 errors.Is(err, os.ErrNotExist) 判断文件是否存在——即使 err 被多层 %w 包装也能命中;As 取出"链中某个具体类型的错误实例":如 var te *TimeoutErr; errors.As(err, &te) 后能访问 te.Op 字段做业务处理。规则:比较哨兵用 Is,需要访问类型字段用 As。

Q 6. 一个 goroutine 里 panic,另一个 goroutine 能用 recover 接住吗?生产上怎么防?"

✅ 参考答案

不能。recover 只能恢复"本 goroutine"的 panic;其他 goroutine 的 panic 会直接终止整个进程(Go 的哲学:panic 表示状态已不可信)。生产防护:每个 goroutine 入口的函数(如 HTTP handler、worker 主循环)用 defer + recover 包一层,把 panic 转为错误日志或 500 响应;框架(如 net/http 服务端、Gin)已内置此机制。

Q 7. context.WithTimeout 之后忘记 defer cancel() 会怎样?

✅ 参考答案

超时时间到之前,该 context 的内部计时器与相关资源不会被释放;在循环中反复创建(如每个请求一个)而忘记 cancel,会积累大量未释放的定时器——这是 goroutine/内存泄漏的来源之一。铁律:WithCancel/WithTimeout 创建后立刻 defer cancel(),即使确认不会取消也要养成习惯。

章末实验题:并发任务调度器

实验 9:工作池 + context 取消 + 竞争检测

实验目标:实现一个生产形态的任务调度器:固定 worker 池消费任务队列,支持超时取消、优雅关闭、统计与竞争检测,覆盖本章全部知识点。

实验要求:

- 任务 1:固定数量 worker 的 goroutine 池,从任务通道消费(覆盖:\$9.1 goroutine、\$9.2 channel)
- 任务 2:任务通道由调度器唯一负责关闭,关闭后 worker 正常退出(覆盖:\$9.3 关闭规范)
- 任务 3:主流程用 WaitGroup 等待全部 worker 结束(覆盖:\$9.5 WaitGroup)
- 任务 4:每个任务模拟耗时(随机 sleep),用 select + time.After 做任务级超时(覆盖:\$9.4 select)
- 任务 5:ctx 取消时,调度器与 worker 全部停止(覆盖:\$9.9 context)
- 任务 6:完成计数用 atomic 累加,统计用 Mutex 保护(覆盖:\$9.5 atomic/Mutex)
- 任务 7:某个任务 panic 时该 worker 不崩、调度器继续(覆盖:\$9.8 recover)
- 任务 8: `go run -race main.go` 无任何告警(覆盖:\$9.6 race)
- 任务 9:任务错误用 %w 包装并 errors.Is 判断汇总(覆盖:\$9.7 错误链)

实验环境:Go 1.21+;先删掉任意一处 Mutex/atomic 保护,运行 `-race` 观察告警,再恢复。

第一步 定义任务类型 Task(带 ID)与"任务错误"哨兵 ErrTaskFailed

第二步 调度器:创建 jobs 通道与 ctx;N 个 worker goroutine 消费;生产者往 jobs 发任务

第三步 任务处理函数 work:模拟耗时,随机失败返回错误;用 defer+recover 包住 panic

第四步 统计结构体 Stats 用 Mutex 保护;成功/失败计数用 atomic.Int64

第五步 关闭:生产者发完任务后 close(jobs);worker 从 jobs 的 for range 自然退出;wg.Wait() 汇总

第六步 取消:context.WithTimeout 传入 worker;work 内 select ctx.Done() 提前返回

✔ 参考答案要点

```
// 实验 9:并发任务调度器(工作池 + context 取消 + 竞争检测)
package main

import (
    "context"
    "errors"
    "fmt"
    "math/rand"
    "sync"
    "sync/atomic"
    "time"
)

// 任务与错误定义($9.7 哨兵错误)
var ErrTaskFailed = errors.New("任务执行失败")

type Task struct {
    ID    int
    Load int // 模拟工作量
}

// Stats 统计信息:Mutex 保护非原子字段($9.5)
type Stats struct {
    mu      sync.Mutex
    Retried int
    Panicked int
    Failed  atomic.Int64 // 原子计数免锁
    Done    atomic.Int64
}

// work 执行单个任务:模拟耗时与随机失败($9.7 错误链)
func work(ctx context.Context, t Task, st *Stats) error {
    if t.Load%7 == 0 { // 故意制造 panic 场景($9.8)
        panic("任务内部异常")
    }
    select {
    case <-time.After(time.Duration(rand.Intn(50)) * time.Millisecond):
        // 模拟业务:10% 概率失败
        if t.Load%10 == 0 {
            st.Failed.Add(1)
            return fmt.Errorf("任务 %d: %w", t.ID, ErrTaskFailed)
        }
        st.Done.Add(1)
        return nil
    case <-ctx.Done():
        st.mu.Lock()
        st.Retried++
        st.mu.Unlock()
        return ctx.Err() // 被取消:不算失败
    }
}

// worker 消费者:循环取任务执行($9.1 $9.2)
func worker(ctx context.Context, id int, jobs <-chan Task, st *Stats, wg *sync.WaitGroup) {
    defer wg.Done() // 任务 3:结束信号
```

```

for t := range jobs { // 通道关闭自动退出($9.3)
    func() {
        defer func() { // 任务 7:panic 不扩散($9.8)
            if r := recover(); r != nil {
                st.mu.Lock()
                st.Panicked++
                st.mu.Unlock()
                fmt.Printf("worker%d 捕获 panic:%v\n", id, r)
            }
        }()
        if err := work(ctx, t, st); err != nil {
            fmt.Printf("worker%d 任务 %d 结束:%v\n", id, t.ID, err)
        }
    }()
}

func main() {
    const workers = 8
    const tasks = 200

    jobs := make(chan Task, 64) // 有缓冲任务队列($9.2)
    ctx, cancel := context.WithTimeout(context.Background(), 3*time.Second)
    defer cancel() // 任务 5+9.9:超时取消

    var st Stats
    var wg sync.WaitGroup

    for i := 0; i < workers; i++ {
        wg.Add(1) // 必须在 goroutine 启动前 Add($9.5)
        go worker(ctx, i, jobs, &st, &wg)
    }

    // 生产者:发任务,发完由唯一发送方关闭($9.3)
    for i := 0; i < tasks; i++ {
        select {
        case jobs <- Task{ID: i, Load: rand.Intn(20)}:
        case <-ctx.Done():
            fmt.Println("任务发送被取消,停止投递")
            break // 注意:仅退出 select;配合标记位可彻底终止
        }
        if ctx.Err() != nil {
            break
        }
    }
    close(jobs) // 关闭义务在生产方

    wg.Wait() // 任务 3:等待所有 worker 结束
    fmt.Println("==== 调度结果 =====")
    fmt.Println("完成:", st.Done.Load())
    fmt.Println("失败:", st.Failed.Load())
    fmt.Println("被取消:", st.Retried) // Mutex 保护($9.5)
    fmt.Println("panic 拦截:", st.Panicked)

    // 结果错误汇总演示:把累计失败数包装为错误链($9.7)

```

```
if st.Failed.Load() > 0 {
    err := fmt.Errorf("调度结束: 有 %d 个任务失败: %w",
        st.Failed.Load(), ErrTaskFailed)
    if errors.Is(err, ErrTaskFailed) {
        fmt.Println("错误链确认:ErrTaskFailed 在链中")
    }
}
fmt.Println("=== go run -race main.go 应无任何告警 ===")
}
```

设计说明:实验代码是本章知识点的总装配;jobs 通道由唯一发送方关闭,worker 以 for range 消费并自然退出——关闭纪律与消费模板的实战形态;Stats 同时演示 Mutex(Retried/Panicked)与 atomic(Failed/Done)两种保护的选择逻辑;ctx 同时作用于任务执行与任务投递两个方向,超时后系统整体收敛;recover 包在匿名函数里确保"单个任务崩溃不影响 worker 存活";最后的错误链演示 errors.Is 的典型用法。改动建议:把 workers 改成 64、tasks 改成 10000 观察吞吐;把 Mutex 换成 atomic 对比语义;尝试把 jobs 换成无缓冲通道观察生产者阻塞行为。

下一章预告:第10章 泛型与元编程。并发是 Go 的"A 面",泛型是 B 面:1.18 引入的类型参数机制——可比较约束、any 约束、泛型函数与类型、slices/maps 标准工具包,以及"Go 没有宏"的元编程哲学。